

Elaborate SRS(Software Requirements Specification) for FLFP: Forward Looking Finance Portfolio

Document Version: 1.0.0

Status: Draft

Date: July 30, 2026

Author / Engineering Lead: Bebandh Shrivastava

Target Architecture: Java Spring Boot, Angular, MySQL (Modular Monolith)

Executive Summary & Project Statement

Forward-Looking Finance Portfolio (FLFP) is an enterprise-grade predictive wealth simulation application. Unlike traditional personal finance platforms that focus on historical spend analysis, FLFP operates as a forward-looking financial forecasting engine.

By combining baseline account balances with user-configured recurring cash flows (credits and expenses), instant transactions, and compound growth vehicles (SIPs and lumpsum investments), FLFP dynamically calculates and projects real-time portfolio liquidity for any chosen future calendar date.

Metadata & Repository References

- **Project Repository:** github.com/[org]/flfp-monolith
- **Architecture Style:** Modular Monolith (Package-by-Feature)
- **Primary Contacts:** Bebandhshrivastava5757@gmail.com

This document is intended for:

- **Backend Engineers:** For implementing Spring Boot REST APIs, package-by-feature domain logic, and JPA mappings.
- **Frontend Engineers:** For building Angular feature modules, RxJS state stores, and dynamic date-picker simulations.
- **Database Administrators:** For reviewing MySQL relational constraints, indexing strategies, and transactional isolation levels.
- **QA / Test Engineers:** For writing end-to-end test suites covering complex edge cases (e.g., month-end calendar overshoots, leap years, precision rounding).

Table of Contents

1. INTRODUCTION

- 1.1 Purpose & Vision
- 1.2 Scope of the Forward-Looking Finance Portfolio (FLFP)
- 1.3 Definitions, Acronyms, and Abbreviations
- 1.4 Tech Stack & Architecture Overview (Spring Boot, Angular, MySQL)

2. SYSTEM ARCHITECTURE & MODULAR MONOLITH DESIGN

- 2.1 High-Level Architecture Diagram & System Context
- 2.2 Spring Boot Package-by-Feature Structure
- 2.3 Database Entity-Relationship (ER) Schema & Constraints
- 2.4 Angular Frontend Architecture & State Management Patterns

3. FUNCTIONAL REQUIREMENTS

- 3.1 Account Management (Multi-Account Context, Balance Baselines)
- 3.2 Expense Management Engine (Recurring vs. Instant Expenses)
- 3.3 Credit & Income Flow Engine (Scheduled Inflows)
- 3.4 Growth & Investment Engine (SIP vs. Lumpsum, Compound Yield Mechanics)
- 3.5 Dynamic Time-Series Projection Engine (Core Simulation Logic)
- 3.6 Analytics, Filters, and Trend Analysis (52-Week Moving Averages)

4. EXTERNAL INTERFACE & API SPECIFICATIONS

- 4.1 REST API Contracts & Endpoint Matrix
- 4.2 API Data Transfer Objects (DTOs) and Validation Rules
- 4.3 Dynamic Projection Request/Response Models
- 4.4 Angular-to-Spring Boot HTTP Interceptor & Error Contracts

5. COMPLEX BUSINESS LOGIC & EDGE CASE SPECIFICATION

5.1 Date & Time Edge Cases (Leap Years, Month-End Overshoot: Jan 31 -> Feb 28)

5.2 Financial Precision & Rounding Rules (BigDecimal vs. Double)

5.3 Transactional Consistency Strategy (Instant Expense Updates vs. Projections)

5.4 Historical Balance Overrides & Retroactive Edits

6. NON-FUNCTIONAL REQUIREMENTS (NFRs)

6.1 Performance & Latency Targets (Sub-50ms Projection Engine Execution)

6.2 Scalability & Database Indexing Strategy

6.3 Security & Data Protection (Auth, CSRF, XSS, Input Sanitization)

6.4 Maintainability & Code Quality Guardrails

7. APPENDIX & REVISION HISTORY

1. INTRODUCTION

1.1 Purpose & Vision

The purpose of this Software Requirements Specification (SRS) is to define the definitive functional, non-functional, and architectural requirements for the **Forward-Looking Finance Portfolio (FLFP)**.

Vision Statement

Traditional personal finance tools focus almost exclusively on historical analytics—explaining where money was spent after the fact. FLFP shifts the paradigm from historical auditing to **predictive wealth simulation**. By modeling an individual's financial ecosystem as a deterministic system of baseline balances, cash-flow cadences (credits and expenses), and investment yields, FLFP empowers users to answer a critical question: *"What will my precise liquid balance be on any given date in the future?"*

1.2 Scope of the Forward-Looking Finance Portfolio (FLFP)

FLFP is designed as a single-tenant or multi-account Web Application built on a modern full-stack architecture.

In-Scope Capabilities

- **Multi-Account Context Management:** Independent tracking of separate real-world financial accounts (e.g., Checking, Savings, Cash Wallets) maintained under a unified user profile.
- **Deterministic Cash-Flow Modeling:** Support for recurring income (credits) and recurring expenses operating across discrete temporal cadences (Daily, Weekly, Monthly, Annual).
- **Instant Expense Ledger:** One-off, unscheduled transactions that immediately update the core real account balance within a strict ACID-compliant database transaction block.
- **Investment Yield Projection Engine:** Simulation of growth vehicles including regular Systematic Investment Plans (SIPs) and Lumpsum deposits (Fixed Deposits, Equity, Real Estate) utilizing compound growth formulas up to maturity or target dates.
- **Dynamic Time-Series Projection:** A real-time simulation engine that projects liquid portfolio balances for any selected future calendar date ($T_{target} > T_{today}$) or auto-calculated standard baselines (+30 Days, +365 Days).
- **Analytics & Trend Metrics:** Aggregated rolling trend metrics, including 52-week moving averages across accounts and expense categories.

Out-of-Scope (Version 1.0)

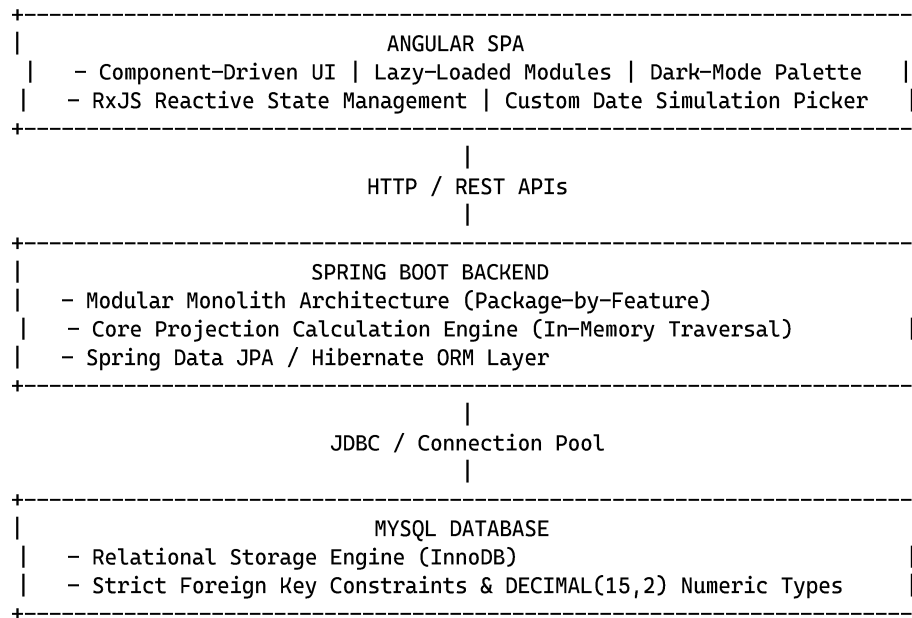
- Direct Third-Party Bank API Integrations (e.g., Plaid, Open Banking OAuth flows).
- Automated tax bracket calculations and region-specific capital gains tax withholding.
- Multi-currency conversion engines (V1.0 operates strictly in a single, user-configured base currency).

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
FLFP	Forward-Looking Finance Portfolio
SIP	Systematic Investment Plan—a strategy where fixed amounts are invested at regular monthly intervals.
Instant Expense	A one-off, immediate transaction that reduces the current baseline account balance upon creation.
Recurring Expense	A scheduled financial commitment occurring at defined intervals (Daily, Weekly, Monthly, Annual) used only for forward projections.
Modular Monolith	An architectural design pattern where code is structured into strictly bounded, domain-specific modules within a single deployment unit.
ACID	Atomicity, Consistency, Isolation, Durability—database properties ensuring reliable transaction processing.
DTO	Data Transfer Object—an object used to encapsulate and transfer data between client (Angular) and server (Spring Boot).
RxJS	Reactive Extensions for JavaScript—a library for reactive programming using Observables used inside Angular.

1.4 Tech Stack & Architecture Overview

FLFP utilizes a cohesive full-stack technology stack optimized for low latency, mathematical precision, and transactional integrity.



Stack Components

Frontend (Client Layer)

- **Framework:** Angular (v17+) Single Page Application (SPA).
- **State Management:** Reactive Streams with **RxJS** Observables and Angular Signals for UI reactivity.
- **Styling & Theme:** Typography-first design system utilizing a dark-mode palette: Deep Charcoal (#121212), Mint Green positive accents (#00E676), Crimson Red negative accents (#FF5252), and Electric Indigo interactive controls (#5C6BC0).

Backend (Application Layer)

- **Runtime & Framework:** Java 17/21 LTS with **Spring Boot 3.x**.
- **Architecture Pattern:** **Modular Monolith** using Package-by-Feature structure (com.finance.flfp.account, com.finance.flfp.expense, com.finance.flfp.projection, etc.).
- **Persistence Layer:** Spring Data JPA with Hibernate ORM, utilizing explicit @Transactional boundary definitions for atomic ledger writes.

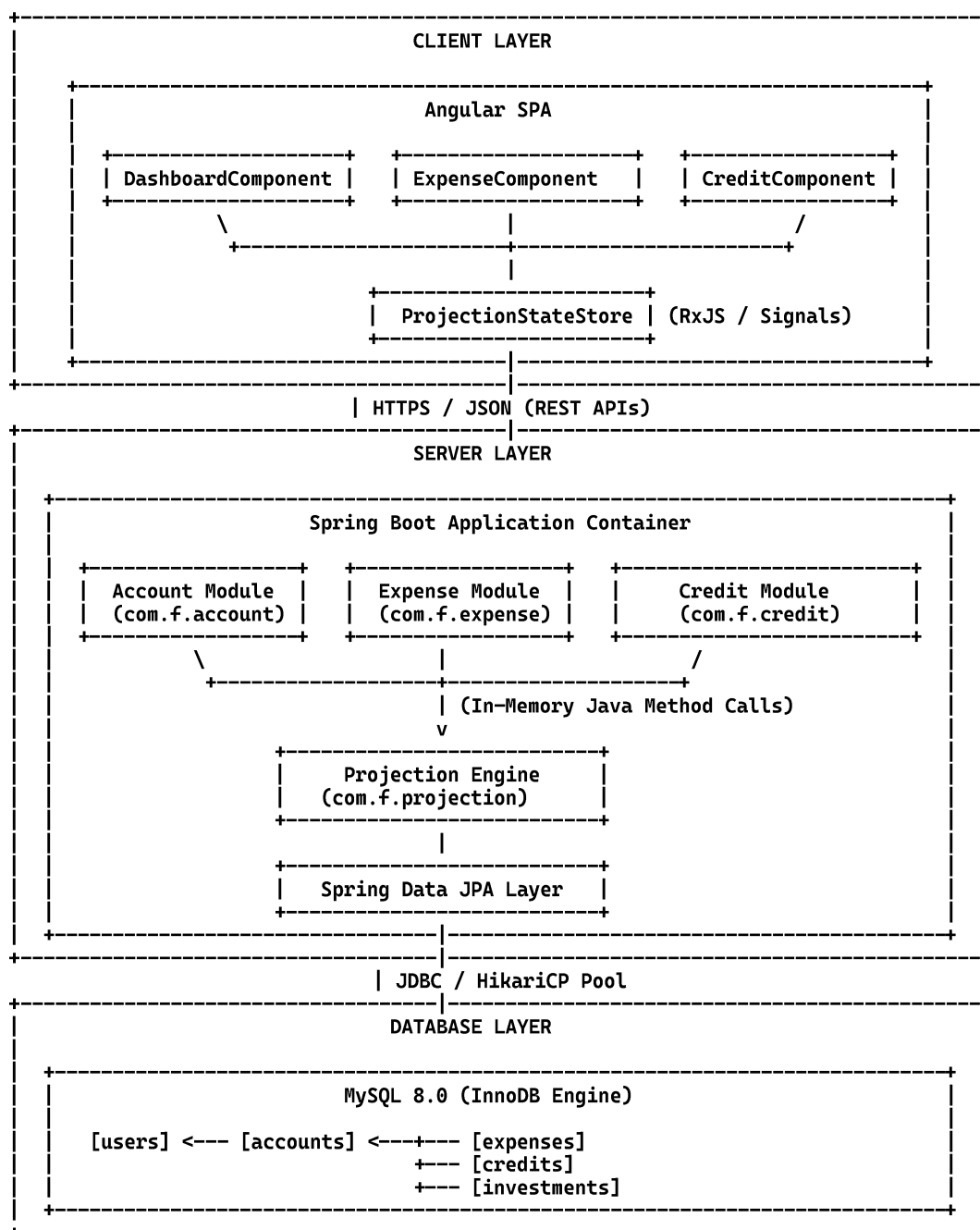
Database (Persistence Layer)

- **Engine:** **MySQL 8.0+** utilizing the InnoDB storage engine for ACID compliance.
- **Numeric Type Standard:** All monetary amounts are strictly typed as DECIMAL(15, 2) to eliminate floating-point rounding errors native to binary standard IEEE 754 representations.

2. SYSTEM ARCHITECTURE & MODULAR MONOLITH DESIGN

2.1 High-Level Architecture Diagram & System Context

The Forward-Looking Finance Portfolio (FLFP) uses a **Modular Monolith Architecture**. Rather than deploying independent microservices over a network—which introduces serialization overhead, distributed tracing complexity, and network latency during complex calculations—FLFP encapsulates all domain boundaries inside a single, highly structured Java Spring Boot runtime backed by a single MySQL database instance.



2.2 Spring Boot Package-by-Feature Structure

To maintain strict domain boundaries and prevent the application from turning into a messy monolithic codebase, the backend codebase is structured **by feature** rather than by technical layer (e.g., placing all controllers in one folder and all models in another).

Each package is self-contained and exposes explicit Public Interfaces (Services/DTOs) for cross-module interactions while keeping internal implementation details package-private.

```
com.finance.flfp
├── FlfpApplication.java
├── account/
│   ├── controller/
│   │   └── AccountController.java
│   ├── dto/
│   │   ├── AccountCreateRequest.java
│   │   └── AccountResponse.java
│   ├── model/
│   │   └── Account.java
│   ├── repository/
│   │   └── AccountRepository.java
│   └── service/
│       └── AccountService.java
├── expense/
│   ├── controller/
│   │   └── ExpenseController.java
│   ├── dto/
│   │   ├── ExpenseCreateRequest.java
│   │   └── ExpenseResponse.java
│   ├── model/
│   │   ├── Expense.java
│   │   ├── ExpenseType.java           // ENUM: RECURRING, INSTANT
│   │   └── RecurrenceInterval.java    // ENUM: DAILY, WEEKLY, MONTHLY, ANNUAL, NONE
│   ├── repository/
│   │   └── ExpenseRepository.java
│   └── service/
│       └── ExpenseService.java        // Coordinates @Transactional instant balance adjustments
├── credit/
│   ├── controller/
│   │   └── CreditController.java
│   ├── dto/
│   │   ├── CreditCreateRequest.java
│   │   └── CreditResponse.java
│   ├── model/
│   │   ├── Credit.java
│   │   └── CreditInterval.java        // ENUM: WEEKLY, MONTHLY, ANNUAL
│   ├── repository/
│   │   └── CreditRepository.java
│   └── service/
│       └── CreditService.java
├── investment/
│   ├── controller/
│   │   └── InvestmentController.java
│   ├── dto/
│   │   ├── InvestmentCreateRequest.java
│   │   └── InvestmentResponse.java
│   ├── model/
│   │   ├── Investment.java
│   │   ├── InvestmentStyle.java       // ENUM: SIP, ONE_TIME
│   │   └── InvestmentType.java        // ENUM: REAL_ESTATE, EQUITY, FIXED_DEPOSIT
│   └── repository/
│       └── InvestmentRepository.java
```



```

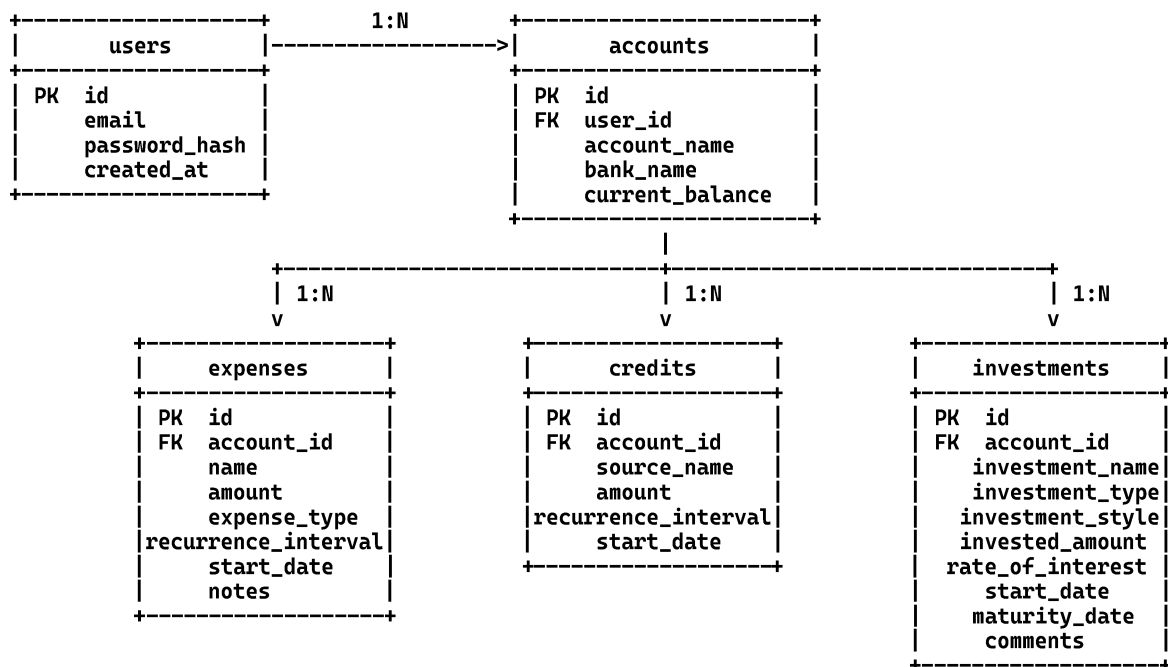
├── service/
│   └── InvestmentService.java
├── projection/
│   ├── controller/
│   │   └── ProjectionController.java
│   ├── dto/
│   │   ├── DashboardSummaryResponse.java
│   │   └── ProjectionResponse.java
│   └── service/
│       ├── ProjectionCalculationEngine.java // Pure calculation logic (no side effects)
│       └── ProjectionService.java           // Orchestrates domain repository fetches
├── shared/
│   ├── config/
│   │   ├── CorsConfig.java
│   │   └── SecurityConfig.java
│   ├── exception/
│   │   ├── GlobalExceptionHandler.java
│   │   └── ResourceNotFoundException.java
│   └── util/
│       └── DateUtils.java

```

2.3 Database Entity-Relationship (ER) Schema & Constraints

The relational schema is configured in MySQL using the InnoDB engine to enforce strict relational integrity, cascading deletions, and exact monetary representation.

Relational Schema Diagram



Complete DDL Script with Constraints and Indexes

```
-- Disable foreign key checks during bootstrap
SET FOREIGN_KEY_CHECKS = 0;

DROP TABLE IF EXISTS investments;
DROP TABLE IF EXISTS credits;
DROP TABLE IF EXISTS expenses;
DROP TABLE IF EXISTS accounts;
DROP TABLE IF EXISTS users;

SET FOREIGN_KEY_CHECKS = 1;

-- 1. USERS TABLE
CREATE TABLE users (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_email_format CHECK (email LIKE '%@%.%')
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

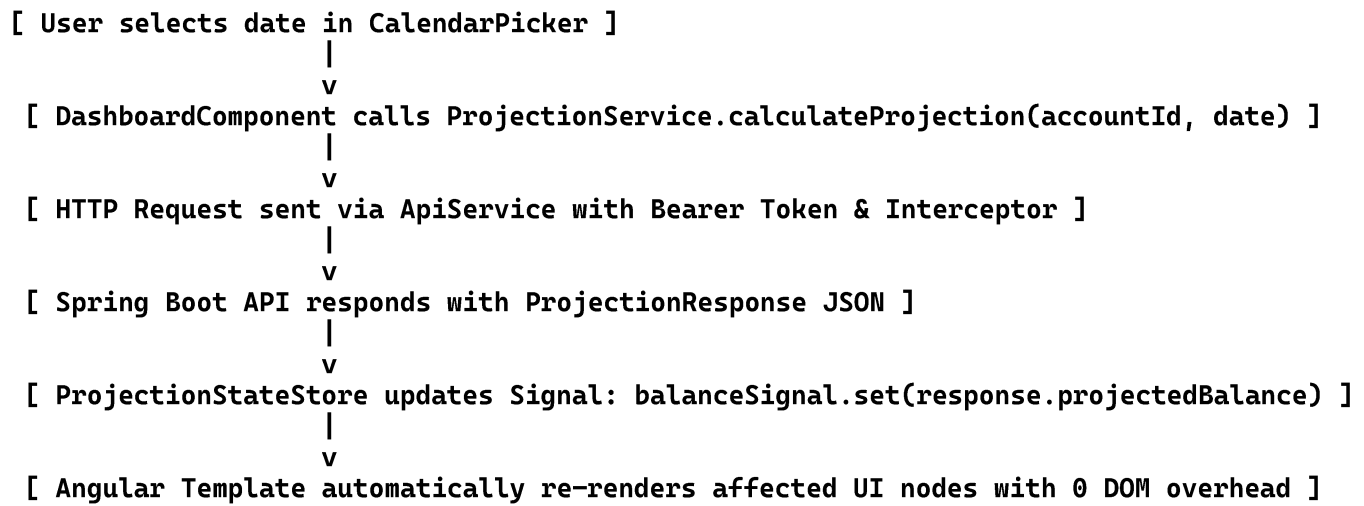
-- 2. ACCOUNTS TABLE
CREATE TABLE accounts (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT NOT NULL,
  account_name VARCHAR(100) NOT NULL,
  bank_name VARCHAR(100) DEFAULT NULL,
  current_balance DECIMAL(15, 2) NOT NULL DEFAULT 0.00,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_accounts_user FOREIGN KEY (user_id)
    REFERENCES users(id) ON DELETE CASCADE,
  INDEX idx_accounts_user (user_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 3. EXPENSES TABLE
CREATE TABLE expenses (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  account_id BIGINT NOT NULL,
  name VARCHAR(100) NOT NULL,
  amount DECIMAL(15, 2) NOT NULL,
  expense_type ENUM('RECURRING', 'INSTANT') NOT NULL,
  recurrence_interval ENUM('DAILY', 'WEEKLY', 'MONTHLY', 'ANNUAL', 'NONE') NOT NULL,
  start_date DATE NOT NULL,
  notes TEXT DEFAULT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_expenses_account FOREIGN KEY (account_id)
    REFERENCES accounts(id) ON DELETE CASCADE,
  CONSTRAINT chk_expense_amount CHECK (amount > 0.00),
  INDEX idx_expenses_account (account_id),
  INDEX idx_expenses_type_date (account_id, expense_type, start_date)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 4. CREDITS TABLE
CREATE TABLE credits (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  account_id BIGINT NOT NULL,
  source_name VARCHAR(100) NOT NULL,
  amount DECIMAL(15, 2) NOT NULL,
  recurrence_interval ENUM('WEEKLY', 'MONTHLY', 'ANNUAL') NOT NULL,
  start_date DATE NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_credits_account FOREIGN KEY (account_id)
    REFERENCES accounts(id) ON DELETE CASCADE,
  CONSTRAINT chk_credit_amount CHECK (amount > 0.00),
  INDEX idx_credits_account (account_id),
  INDEX idx_credits_start_date (account_id, start_date)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 5. INVESTMENTS TABLE
CREATE TABLE investments (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  account_id BIGINT NOT NULL,
  investment_name VARCHAR(100) NOT NULL,
  investment_type ENUM('REAL_ESTATE', 'EQUITY', 'FIXED_DEPOSIT') NOT NULL,
```

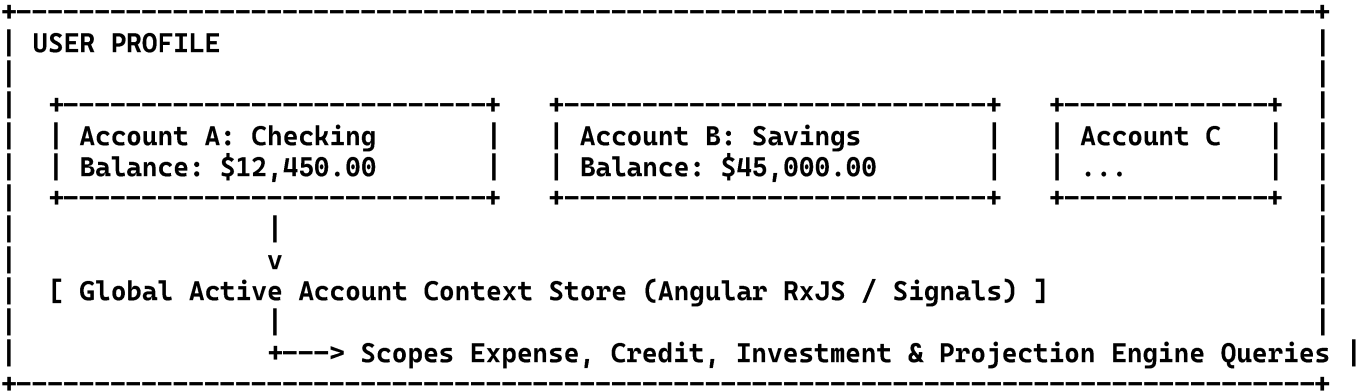

Unidirectional State Management Flow



3. FUNCTIONAL REQUIREMENTS

3.1 Account Management (Multi-Account Context, Balance Baselines)

The Account Management module acts as the root boundary for all financial calculations. The system must support multiple independent accounts under a single user profile.

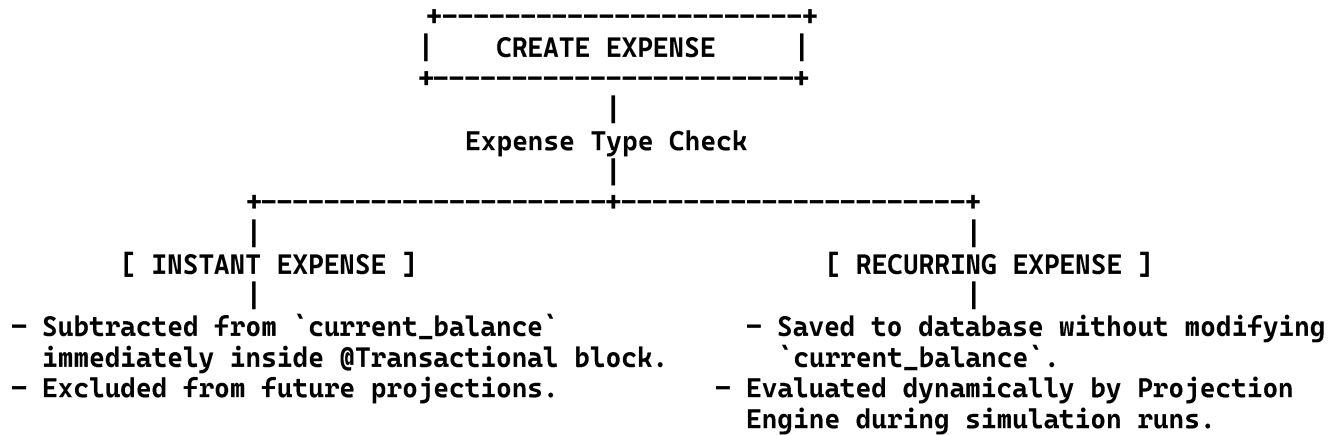


3.1.1 Feature Specifications

- **FR-ACC-01 (Creation & Configuration):** The system shall allow users to create multiple financial accounts by specifying an `account_name` (e.g., "Main Checking", "Emergency Fund"), an optional `bank_name`, and an initial `current_balance`.
- **FR-ACC-02 (Account Selection Context):** The Angular UI shall provide a persistent, global account switcher in the sidebar. Selecting an account updates the application state, filtering all expenses, credits, investments, and projection simulations to that specific account ID.
- **FR-ACC-03 (Baseline Balance Persistence):** The system shall maintain `current_balance` as the single source of truth for liquid funds *as of the present moment* (T_{today}).
- **FR-ACC-04 (Account Deletion & Cascading):** Deleting an account shall trigger a cascading deletion across all associated expenses, credits, and investments within an ACID database transaction.

3.2 Expense Management Engine (Recurring vs. Instant Expenses)

The Expense module classifies cash outflows into two operational categories: **Instant** (realized immediately) and **Recurring** (projected over time).



3.2.1 Feature Specifications

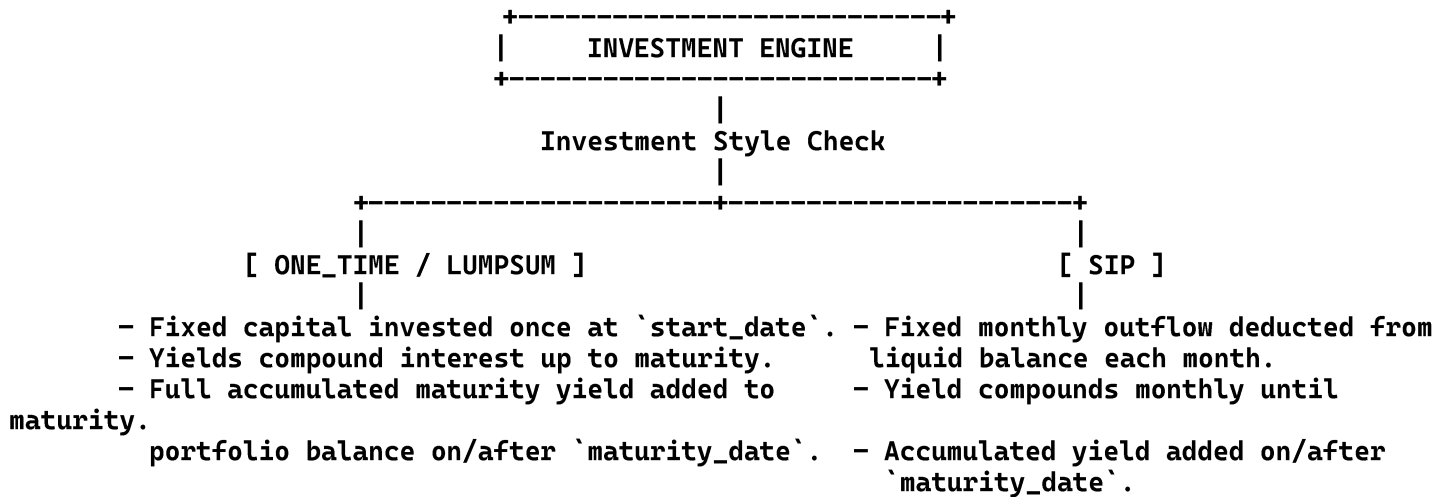
- **FR-EXP-01 (Expense Categorization):** Each expense record must capture name, amount, expense_type (INSTANT or RECURRING), recurrence_interval (DAILY, WEEKLY, MONTHLY, ANNUAL, NONE), and a start_date.
- **FR-EXP-02 (Instant Expense Execution):**
 - When an INSTANT expense is saved, the backend shall immediately deduct the amount from accounts.current_balance.
 - The operation **must** run inside a @Transactional block. If the account balance update fails, the expense creation must rollback completely.
 - Instant expenses are historical markers and **shall be ignored** by the dynamic projection engine to prevent double-counting.
- **FR-EXP-03 (Recurring Expense Rule Enforcing):**
 - RECURRING expenses shall not modify accounts.current_balance upon creation.
 - The system shall enforce that RECURRING expenses have a valid interval (DAILY, WEEKLY, MONTHLY, or ANNUAL).
 - If expense_type == INSTANT, recurrence_interval must automatically default to NONE.

3.3.1 Feature Specifications

- **FR-CRE-01 (Credit Registration):** The system shall record inflows specifying source_name, amount, recurrence_interval (WEEKLY, MONTHLY, ANNUAL), and start_date.
- **FR-CRE-02 (Inflow Recurrence Engine):**
 - Credits are strictly scheduled events and do not increase current_balance on creation.
 - The Projection Engine shall treat credits as positive additives (+) occurring sequentially from start_date up to the selected simulation target date.

3.4 Growth & Investment Engine (SIP vs. Lumpsum, Compound Yield Mechanics)

The Investment module models wealth accumulation vehicles across two styles: **Systematic Investment Plans (SIP)** and **One-Time Lumpsum Deposits** (e.g., Fixed Deposits, Real Estate, Equity).



3.4.1 Feature Specifications

- **FR-INV-01 (Investment Parameter Entry):** The system shall record investments with investment_name, investment_type (REAL_ESTATE, EQUITY, FIXED_DEPOSIT), investment_style (SIP or ONE_TIME), invested_amount, rate_of_interest, start_date, and optional maturity_date.
- **FR-INV-02 (SIP Outflow Modeling):** For SIP investments, the projection engine shall simulate a monthly cash deduction equal to invested_amount from liquid funds starting at start_date up to the simulation target date or maturity_date.
- **FR-INV-03 (Lumpsum Compounding Logic):** For ONE_TIME investments, compound growth is calculated using annual compounding:

$$A = P \times \left(1 + \frac{r}{100}\right)^t$$

Where P is invested_amount, r is rate_of_interest, and t is time in years elapsed from start_date to effective date.

- **FR-INV-04 (SIP Future Value Yield Logic):** For SIP investments, future yield value is calculated using the annuity compounding formula:

$$FV = P \times \left[\frac{(1 + i)^n - 1}{i} \right] \times (1 + i)$$

Where P is monthly contribution, $i = \frac{\text{rate}}{12 \times 100}$, and n is total months elapsed.

- **FR-INV-05 (Maturity Liquidation Event):** The calculated yield (accumulated wealth) from an investment shall **only** be added back to the total projected liquid balance if the simulation target date $T_{target} \geq \text{maturity_date}$.

3.5 Dynamic Time-Series Projection Engine (Core Simulation Logic)

The Projection Engine is the central processing component of FLFP. It dynamically calculates future account balances for any given target date without modifying database state.

3.5.1 Algorithmic Execution Steps

The core projection engine follows a deterministic, sequential traversal algorithm to compute predicted balances. Given an active account A , a baseline execution date T_{today} , and a target evaluation date T_{target} , the simulation pipeline executes through five distinct processing stages:

Stage 1: Baseline Context Initialization

Fetch the target account's current balance $B_{current}$ from the database. Set the initial projected balance state $B_{proj} = B_{current}$.

Evaluate Guard Condition: $T_{target} \leq T_{today} \Rightarrow \text{Return } B_{current}$

Stage 2: Scheduled Inflow Traversal (Credits)

Iterate through all registered credits C linked to the target account. For each credit $c \in C$, compute total occurrences N_c between $c.\text{start_date}$ and T_{target} based on its recurrence cadence (WEEKLY, MONTHLY, ANNUAL):

$$B_{proj} \leftarrow B_{proj} + \sum_{c \in C} (c.\text{amount} \times N_c)$$

Stage 3: Scheduled Outflow Traversal (Recurring Expenses)

Iterate through all expenses E linked to the target account where $e.\text{expense_type} = \text{RECURRING}$. Compute total occurrences N_e between $e.\text{start_date}$ and T_{target} based on its recurrence cadence (DAILY, WEEKLY, MONTHLY, ANNUAL):

$$B_{proj} \leftarrow B_{proj} - \sum_{e \in E_{\text{recurring}}} (e.\text{amount} \times N_e)$$

Note: Instant expenses ($e.\text{expense_type} = \text{INSTANT}$) are strictly filtered out during Stage 3 traversal as their financial impact is applied directly to $B_{current}$ upon transaction creation.

Stage 4: Wealth Vehicle Outflow & Maturity Traversal (Investments)

Iterate through all investment assets I linked to the target account to simulate liquid capital contributions and asset maturity liquidations:

1. Systematic Investment Plan (SIP) Deductions:

If $i.investment_style = SIP$, determine the effective evaluation limit $T_{eval} = \min(T_{target}, i.maturity_date)$. Calculate total monthly SIP payment occurrences N_{sip} between $i.start_date$ and T_{eval} :

$$B_{proj} \leftarrow B_{proj} - (i.invested_amount \times N_{sip})$$

2. Maturity Yield Realization:

If $i.maturity_date \neq NULL$ and $T_{target} \geq i.maturity_date$, calculate the accumulated compound yield Y_i as of the maturity date and add the payout to liquid reserves:

$$B_{proj} \leftarrow B_{proj} + Y_i$$

Stage 5: Precision Normalization & Response Construction

Format the final projected value B_{proj} to enforce financial currency precision using Java's `RoundingMode.HALF_UP`:

$$B_{final} = \frac{\lfloor B_{proj} \times 10^2 + 0.5 \rfloor}{10^2}$$

$$\Delta = B_{final} - B_{current}$$

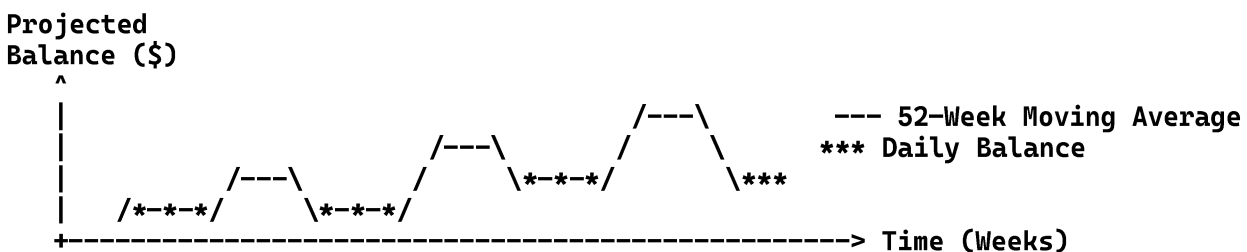
Construct and return the `ProjectionResponse` DTO containing `accountId`, `targetDate`, `projectedBalance` (B_{final}), and `deltaVariance` (Δ).

3.5.2 Feature Specifications

- **FR-PRJ-01 (Dashboard Summary Baseline):** The system shall expose a summary endpoint returning simulated balances for **+30 Days** and **+365 Days** from T_{today} .
- **FR-PRJ-02 (Arbitrary Calendar Date Simulation):** The Angular date picker shall allow selecting any valid future date up to **+50 Years**. The engine must compute and return the projected balance with sub-50ms latency.
- **FR-PRJ-03 (Delta Variance Metric):** Projection responses shall return both the total projected balance and the net change (Δ) relative to `current_balance`.

3.6 Analytics, Filters, and Trend Analysis (52-Week Moving Averages)

The Analytics module provides analytical views over projected time-series data to help users visualize trends and financial velocity.



3.6.1 Feature Specifications

- FR-ANL-01 (Interval Moving Average Calculation):** The system shall generate a 52-week moving average (SMA_{52}) for projected balances by sampling weekly balance projections over a rolling 52-week window:

$$SMA_{52} = \frac{1}{52} \sum_{i=1}^{52} B_{week_i}$$

- FR-ANL-02 (Category Filtering):** Users shall be able to filter expense and credit analytics by date range, recurrence type, and category tags.
- FR-ANL-03 (Outflow Velocity Index):** The system shall compute a monthly burn-rate metric summarizing total recurring obligations due over any 30-day window.

4. EXTERNAL INTERFACE & API SPECIFICATIONS

4.1 REST API Contracts & Endpoint Matrix

All API endpoints follow RESTful conventions under the base path `/api/v1`. Requests and responses consume and produce `application/json` data using UTF-8 encoding.

Domain	HTTP Method	Endpoint Path	Description	Expected Status Code
Account	POST	<code>/api/v1/accounts</code>	Create a new financial account	201 Created
Account	GET	<code>/api/v1/accounts/user/{userId}</code>	Fetch all accounts for a user	200 OK
Account	GET	<code>/api/v1/accounts/{accountId}</code>	Fetch account details by ID	200 OK
Account	DELETE	<code>/api/v1/accounts/{accountId}</code>	Delete account (Cascades to child ledger entries)	204 No Content

Domain	HTTP Method	Endpoint Path	Description	Expected Status Code
Expense	POST	/api/v1/expenses	Create an expense (Instant updates account balance)	201 Created
Expense	GET	/api/v1/expenses/account/{accountId}	List all expenses for an account	200 OK
Expense	DELETE	/api/v1/expenses/{expenseId}	Delete an expense entry	204 No Content
Credit	POST	/api/v1/credits	Create a scheduled income/credit entry	201 Created
Credit	GET	/api/v1/credits/account/{accountId}	List all credits for an account	200 OK
Credit	DELETE	/api/v1/credits/{creditId}	Delete a credit entry	204 No Content
Investment	POST	/api/v1/investments	Create a SIP or One-Time investment asset	201 Created
Investment	GET	/api/v1/investments/account/{accountId}	List all investments for an account	200 OK
Investment	DELETE	/api/v1/investments/{investmentId}	Delete an investment entry	204 No Content

Domain	HTTP Method	Endpoint Path	Description	Expected Status Code
Projection	GET	/api/v1/projections/dashboard-summary	Get baseline +30D and +365D metrics	200 OK
Projection	GET	/api/v1/projections/calculate-date	Compute simulated balance for target date	200 OK

4.2 API Data Transfer Objects (DTOs) and Validation Rules

Data validation is strictly enforced at the API controller level using Spring Boot Validation (jakarta.validation.constraints). Invalid payloads trigger a 400 Bad Request with structured field errors.

4.2.1 Account DTOs

AccountCreateRequest

JSON

```
{
  "userId": 1,
  "accountName": "Primary Checking",
  "bankName": "Chase Bank",
  "currentBalance": 12500.50
}
```

- **Validation Rules:**

- `userId: @NotNull`
- `accountName: @NotBlank, @Size(max = 100)`
- `bankName: @Size(max = 100)`
- `currentBalance: @NotNull, @DecimalMin(value = "0.00", message = "Initial balance cannot be negative")`

4.2.2 Expense DTOs

ExpenseCreateRequest

JSON

```
{
  "accountId": 10,
  "name": "Apartment Rent",
  "amount": 2200.00,
  "expenseType": "RECURRING",
  "recurrenceInterval": "MONTHLY",
  "startDate": "2026-08-01",
  "notes": "Monthly rent payment due on 1st"
}
```

- **Validation Rules:**

- accountId: @NotNull
- name: @NotBlank, @Size(max = 100)
- amount: @NotNull, @DecimalMin(value = "0.01", message = "Expense amount must be greater than zero")
- expenseType: @NotNull (**Enum:** RECURRING, INSTANT)
- recurrenceInterval: @NotNull (**Enum:** DAILY, WEEKLY, MONTHLY, ANNUAL, NONE)
- startDate: @NotNull, @NotNull format ISO-8601 (yyyy-MM-dd)

4.2.3 Credit DTOs

CreditCreateRequest

JSON

```
{
  "accountId": 10,
  "sourceName": "Software Engineer Salary",
  "amount": 6500.00,
  "recurrenceInterval": "MONTHLY",
  "startDate": "2026-08-01"
}
```

- **Validation Rules:**

- accountId: @NotNull
- sourceName: @NotBlank, @Size(max = 100)
- amount: @NotNull, @DecimalMin(value = "0.01")
- recurrenceInterval: @NotNull (**Enum:** WEEKLY, MONTHLY, ANNUAL)
- startDate: @NotNull

4.2.4 Investment DTOs

InvestmentCreateRequest

JSON

```
{
  "accountId": 10,
  "investmentName": "S&P 500 Index SIP",
  "investmentType": "EQUITY",
  "investmentStyle": "SIP",
  "investedAmount": 500.00,
  "rateOfInterest": 10.50,
  "startDate": "2026-08-01",
  "maturityDate": "2036-08-01",
  "comments": "Long term wealth compounding"
}
```

```
}
```

- **Validation Rules:**

- accountId: @NotNull
- investmentName: @NotBlank, @Size(max = 100)
- investmentType: @NotNull (**Enum:** REAL_ESTATE, EQUITY, FIXED_DEPOSIT)
- investmentStyle: @NotNull (**Enum:** SIP, ONE_TIME)
- investedAmount: @NotNull, @DecimalMin(value = "0.01")
- rateOfInterest: @NotNull, @DecimalMin(value = "0.00"), @DecimalMax(value = "100.00")
- startDate: @NotNull
- maturityDate: **Optional** (Must be equal to or after startDate if provided)

4.3 Dynamic Projection Request/Response Models

4.3.1 Dashboard Summary Query

- **Endpoint:** GET /api/v1/projections/dashboard-summary?accountId=10

DashboardSummaryResponse

JSON

```
{
  "accountId": 10,
  "currentBalance": 12500.50,
  "projectedBalance30Days": 16300.50,
  "projectedBalance1Year": 58100.00,
  "calculatedAt": "2026-07-30T13:02:44Z"
}
```

4.3.2 Target Calendar Date Simulation Query

- **Endpoint:** GET /api/v1/projections/calculate-date?accountId=10&targetDate=2027-12-31

ProjectionResponse

JSON

```
{
  "accountId": 10,
  "targetDate": "2027-12-31",
  "projectedBalance": 103700.25,
  "deltaVariance": 91199.75,
  "calculatedAt": "2026-07-30T13:02:44Z"
}
```

4.4 Angular-to-Spring Boot HTTP Interceptor & Error Contracts

4.4.1 Standardized Error Payload Schema

All uncaught exceptions or validation failures returned by the Spring Boot backend adhere to RFC 7807 Problem Details standard:

JSON

```
{
  "timestamp": "2026-07-30T13:02:44Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed for request payload",
  "path": "/api/v1/expenses",
  "fieldErrors": [
    {
      "field": "amount",
      "rejectedValue": -50.00,
      "message": "Expense amount must be greater than zero"
    }
  ]
}
```

4.4.2 Angular Error Handling Interceptor Architecture

The Angular application implements a functional `HttpInterceptor` (`error-handler.interceptor.ts`) to intercept failed HTTP responses, transform errors into user-friendly UI notifications, and maintain clean component code.

[Angular HTTP Client]



[HttpRequest Sent to Backend]



[HTTP Response 2xx] → Returns Observable Data



[HTTP Response 4xx/5xx]



[ErrorHandlerInterceptor]



→ 401 Unauthorized → Redirect to /login

→ 400 Bad Request → Display Form Field Errors

→ 404 Not Found → Show Toast ("Resource not found")

→ 500 Server Error → Show Toast ("Simulation service

unavailable")

Angular Interceptor Implementation Code

TypeScript

```
import { HttpInterceptorFn, HttpErrorResponse } from '@angular/common/http';
import { inject } from '@angular/core';
import { Router } from '@angular/router';
import { catchError, throwError } from 'rxjs';
import { NotificationService } from '../services/notification.service';

export const errorHandlerInterceptor: HttpInterceptorFn = (req, next) => {
  const router = inject(Router);
  const notificationService = inject(NotificationService);

  return next(req).pipe(
```

```

catchError((error: HttpErrorResponse) => {
  let errorMessage = 'An unexpected error occurred.';

  if (error.error instanceof ErrorEvent) {
    // Client-side or network error
    errorMessage = `Client Error: ${error.error.message}`;
  } else {
    // Backend HTTP error response
    switch (error.status) {
      case 400:
        errorMessage = error.error?.message || 'Invalid request parameters.';
        break;
      case 401:
        errorMessage = 'Session expired. Please log in again.';
        router.navigate(['/login']);
        break;
      case 404:
        errorMessage = error.error?.message || 'Requested resource not found.';
        break;
      case 500:
        errorMessage = 'Internal server error. Projection engine temporarily
unavailable.';
        break;
      default:
        errorMessage = `Server Error (${error.status}): ${error.statusText}`;
    }
  }

  notificationService.showError(errorMessage);
  return throwError(() => new Error(errorMessage));
})
);

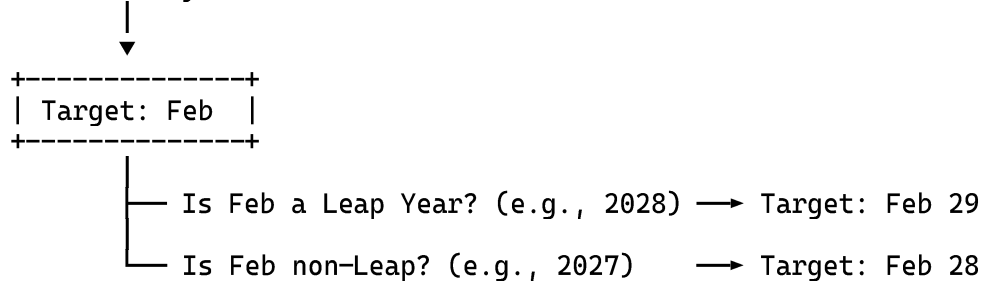
```


5. COMPLEX BUSINESS LOGIC & EDGE CASE SPECIFICATION

5.1 Date & Time Edge Cases

Financial projections spanning calendar intervals face boundary conditions, varying month lengths, and leap years. The simulation algorithm resolves these temporal anomalies deterministically.

[Jan 31st Monthly Recurrence]



5.1.1 Month-End Overshoot Resolution

- **Problem:** A recurring expense or credit set to recur on the 29th, 30th, or 31st of a month encounters a target month with fewer days (e.g., a \$2,000 monthly credit created on January 31 evaluated for February).
- **Rule:** If the start date's day-of-month D_{start} exceeds the maximum valid day of the target month $D_{\text{max}}(M_{\text{target}}, Y_{\text{target}})$, the system clamps the occurrence date to the **last day of that target month**.
- **Formula:**

$$\text{Effective Day} = \min(D_{\text{start}}, D_{\text{max}}(M_{\text{target}}, Y_{\text{target}}))$$

5.1.2 Leap Year Handling (Feb 29)

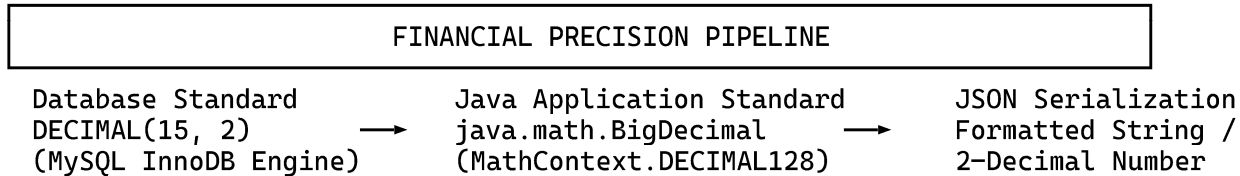
- **Problem:** Annual or monthly occurrences initiated on February 29 during a leap year (e.g., 2028) must be evaluated in non-leap years (e.g., 2029, 2030).
- **Rule:** In non-leap years, February 29 occurrences automatically evaluate on **February 28**. In subsequent leap years, the occurrence reverts to February 29.

5.1.3 Timezone Normalization

- **Problem:** Server-client timezone offsets (e.g., UTC vs. IST/EST) causing date shifting across midnight during dynamic calculations.
- **Rule:** All database start_date, maturity_date, and request targetDate parameters are stored and processed strictly as **ISO-8601 calendar dates (yyyy-MM-dd) without time or offset components**. Calculations use java.time.LocalDate on the backend to guarantee timezone independence.

5.2 Financial Precision & Rounding Rules

Floating-point arithmetic introduces silent precision decay over long time horizons. FLFP enforces floating-point safety across the entire application stack.



5.2.1 Mandatory Type Enforcement

- **Java Backend:** Under no circumstances shall primitive float, double, or wrapper java.lang.Double types be used for financial representations. All monetary values, interest rates, and yield totals use java.math.BigDecimal.
- **Database Schema:** All monetary columns use DECIMAL(15, 2), allowing representation of balances up to 999,999,999,999.99 (10^{11} scale) while strictly preventing binary floating-point roundoff errors.

5.2.2 Compound Interest Computation Scaling

- **Intermediate Calculations:** During exponential calculation steps (SIP compound growth, compound interest), intermediate operations retain **12 places of decimal precision** (MathContext.DECIMAL128).
- **Final Presentation Scaling:** Final calculated balances scale down to **2 decimal places** using RoundingMode.HALF_UP (Round Half Toward "Nearest Neighbor"):

$\text{Scale}(V) = \text{BigDecimal}(V).\text{setScale}(2, \text{RoundingMode.HALF_UP})$

5.3 Transactional Consistency & Concurrency Control

Updating baseline account balances while simultaneously executing reading queries creates potential race conditions.

[Client Request: Instant Expense \$150.00]



[@Transactional Boundary Initiated]



1. Acquire PESSIMISTIC_WRITE lock on target Account
2. Verify account exists and lock row
3. Execute: `balance = balance - 150.00`
4. Insert Expense record (type = INSTANT)
5. Commit Transaction & Release Lock

5.3.1 Instant Expense Atomicity

- Creating an INSTANT expense requires deducting the expense amount directly from `accounts.current_balance`.
- **Isolation Enforcement:** The operation executes within a Spring Boot `@Transactional(isolation = Isolation.READ_COMMITTED)` boundary.
- **Row-Level Locking:** To prevent lost updates during concurrent requests, the account balance update uses explicit row locking (`SELECT ... FOR UPDATE` via Spring Data JPA `@Lock(LockModeType.PESSIMISTIC_WRITE)`).

5.3.2 Projection Engine Read Consistency

- The projection calculation engine runs within `@Transactional(readOnly = true)`. It performs **in-memory simulation traversal** without writing state or acquiring write locks, maintaining sub-50ms execution speed without blocking concurrent transactional updates.

5.4 Retroactive Edits & Historical Balance Overrides

Users may adjust initial account balances or edit historical transaction start dates after projection data has been rendered.

5.4.1 Retroactive Ledger Modification Matrix

Modified Attribute	Execution Impact	State Reconciliation Logic
current_balance Override	Baseline Shift	Modifies $B_{current}$ immediately. All future projection simulations recalculate from the new baseline on subsequent API calls.
start_date Shift (Future Date)	Delayed Impact	Recalculates total occurrence count N . Occurrences prior to the new <code>start_date</code> are excluded from future calculations.
start_date Shift (Past Date)	Historical Inclusion	If new <code>start_date</code> $\leq T_{today}$, occurrence counting begins from <code>start_date</code> up to T_{target} .
Instant Expense Deletion	Balance Reversal	Deleting an INSTANT expense triggers an automatic atomic reversal: $B_{current} = B_{current} + expense.amount$.

6. NON-FUNCTIONAL REQUIREMENTS (NFRs)

6.1 Performance & Latency Targets

The core value proposition of FLFP relies on real-time, interactive simulations. As users scrub through the Angular date picker, dynamic balance calculations must feel instantaneous.

LATENCY BUCKETS & SLA TARGETS

HTTP REST Payload Spring Boot Runtime MySQL InnoDB Read
←──────────────────────────────── 50ms (p95) ─────────────────────────────────→

[Dynamic Date Simulation] → Target Engine Latency: < 15ms (In-Memory)
[Dashboard Summary (+30D)] → Target Overall Latency: < 50ms
[Cold App Load / Bootstrap] → Target Overall Latency: < 800ms

6.1.1 Performance Service Level Agreements (SLAs)

- **NFR-PRF-01 (Simulation Calculation Latency):** The backend dynamic projection calculation engine shall compute target date projections and return responses within $\leq$ **50ms at 95th percentile (p95)** for accounts containing up to 500 active financial items (expenses, credits, investments).
- **NFR-PRF-02 (Dashboard Payload Latency):** The baseline summary endpoint (/api/v1/projections/dashboard-summary) shall deliver responses within $\leq$ **100ms (p99)**.
- **NFR-PRF-03 (UI Render Smoothness):** The Angular dashboard shall update all dynamic text nodes and graph visualizers within **16.6ms (maintaining 60 FPS)** following HTTP response delivery.
- **NFR-PRF-04 (Throughput Capacity):** The Spring Boot backend runtime shall handle a minimum of **250 concurrent simulation queries per second (RPS)** per instance under a memory footprint $\leq$ 512MB RAM.

6.2 Scalability & Database Indexing Strategy

Although FLFP uses a Modular Monolith, the database design must prevent query degradation as transaction ledgers grow.

6.2.1 Database Indexing Specification

To guarantee sub-millisecond query execution when fetching account ledger entries for simulation runs, MySQL indexes are optimized specifically for filtering by `account_id` and temporal ranges.

SQL

-- 1. Index for Account-Scoped Ledger Retrieval

```
CREATE INDEX idx_expenses_account_type ON expenses(account_id, expense_type);
```

```
CREATE INDEX idx_credits_account_start ON credits(account_id, start_date);
```

```
CREATE INDEX idx_investments_account_dates ON investments(account_id, start_date, maturity_date);
```

-- 2. Covering Index for Instant Expense Transaction Lookup

```
CREATE INDEX idx_expenses_instant_lookup ON expenses(account_id, expense_type, start_date, amount);
```

6.2.2 Query Execution Optimization Rules

- **NFR-SCL-01 (Index Scan Enforcement):** All Spring Data JPA queries executing projection context loads **must** execute as indexed range scans (type: ref or range in MySQL EXPLAIN). Full table scans (type: ALL) are explicitly prohibited.
- **NFR-SCL-02 (HikariCP Connection Pooling):** The Spring Boot application shall configure HikariCP with optimized connection bounds:

YAML

```
spring:
  datasource:
    hikari:
      minimum-idle: 5
      maximum-pool-size: 20
      idle-timeout: 300000
      max-lifetime: 1800000
      connection-timeout: 20000
```

6.3 Security & Data Protection

Financial application data requires strict confidentiality, request verification, and defensive input sanitization.

SECURITY & PROTECTION LAYERS

[Angular Client]

├── Cross-Site Scripting (XSS) Sanitization (Angular Engine)
├── HTTP Interceptor Attaches CSRF / Bearer Tokens

↓
[HTTPS Transport Layer] → TLS 1.3 Encryption in Transit

↓
[Spring Boot Security Filters]

├── Rate Limiting Filter (Bucket4j: Max 100 req/min per IP)
├── Request Parameter Validation (jakarta.validation)
└── SQL Injection Defense (Hibernate Parameterized Bindings)

6.3.1 Security Specifications

- **NFR-SEC-01 (Transport Layer Encryption):** All communication between client and server must enforce **HTTPS via TLS 1.3**. Non-secure HTTP requests must automatically receive a 301 Moved Permanently redirect to HTTPS.
- **NFR-SEC-02 (Injection & XSS Prevention):**
 - All database operations must utilize Spring Data JPA / Hibernate parameterized queries. Raw native SQL concatenations are strictly forbidden to eliminate **SQL Injection (SQLi)** vulnerabilities.
 - The Angular frontend contextually encodes rendered strings automatically, shielding the client layer against **Cross-Site Scripting (XSS)** attacks.
- **NFR-SEC-03 (Password Hashing Standard):** User credentials must be salted and hashed using **BCrypt** with a configurable work factor (12 rounds minimum) prior to persistence. Plaintext passwords shall never touch logs or storage.
- **NFR-SEC-04 (CSRF & CORS Defense):**
 - REST API endpoints enforce origin checking via Spring Security CORS configurations restricted strictly to authorized client domains.
 - State-modifying requests (POST, PUT, DELETE) incorporate CSRF token verification headers or statutory stateless Bearer Token verification.

6.4 Maintainability & Code Quality Guardrails

To preserve modular monolith boundary isolation and prevent architectural decay over time:

6.4.1 Maintainability Specifications

- **NFR-MNT-01 (Package Boundary Isolation):** Direct inter-package entity cross-referencing is restricted. Domain packages (account, expense, credit, investment) must communicate through dedicated Service APIs or DTO contracts.
- **NFR-MNT-02 (Unit Test Coverage):** The core calculation engine (ProjectionCalculationEngine.java) must maintain $\geq$ **95% line and branch test coverage** verified by JaCoCo test suites during CI/CD build pipelines.
- **NFR-MNT-03 (Zero Floating-Point Violations):** Static code analysis rules (SonarQube / SpotBugs) shall automatically break automated builds if primitive float or double fields are introduced within financial calculation paths.

7. APPENDIX & REVISION HISTORY

7.1 Acronyms and Abbreviations Glossary

Term	Expansion	Context / Definition
ACID	Atomicity, Consistency, Isolation, Durability	Database transaction properties guaranteed by MySQL InnoDB to prevent corrupt state.
API	Application Programming Interface	RESTful HTTP contracts exposing Spring Boot services to the Angular frontend.
CORS	Cross-Origin Resource Sharing	Security mechanism restricting HTTP requests initiated from browser scripts.
CSRF / XSRF	Cross-Site Request Forgery	An attack vector mitigated via stateless Bearer token validation and header enforcement.
DTO	Data Transfer Object	Immutable Java records/classes used to transfer data across network boundaries without exposing JPA entities.
FLFP	Forward-Looking Finance Portfolio	The personal finance wealth simulation platform specified in this SRS.
JPA	Jakarta Persistence API	Java specification for object-relational mapping implemented via Hibernate ORM.
LMS / SIP	Systematic Investment Plan	Monthly regular contributions toward compounding investment vehicles.
NFR	Non-Functional Requirement	System qualities (performance, security, scalability) defining system operation rather than behaviors.
ORM	Object-Relational Mapping	Technique converting data between incompatible type systems in databases and OOP languages.

Term	Expansion	Context / Definition
RPS	Requests Per Second	Throughput capacity metric measuring server workload capability.
RxJS	Reactive Extensions for JavaScript	Library for reactive programming using Observables within Angular applications.
SLA	Service Level Agreement	Target metric bounds governing response times and availability commitments.
SMA	Simple Moving Average	Rolling technical metric calculated over dynamic 52-week balance time-series windows.
SPA	Single Page Application	Web application architecture executed inside the browser (Angular runtime).
SRS	Software Requirements Specification	Formal ISO/IEC/IEEE 29148 compliant technical specifications document.
XSS	Cross-Site Scripting	Vulnerability mitigated via Angular's contextual auto-sanitization template engine.

7.2 System References & Tooling Standards

- **IEEE Standard:** ISO/IEC/IEEE 29148:2018 Systems and software engineering — Life cycle processes — Requirements engineering.
- **Database Driver Standard:** MySQL Connector/J with HikariCP high-performance connection pool management.
- **Java Language Standard:** OpenJDK 17 LTS / 21 LTS executing with -XX:+UseG1GC garbage collection parameters.
- **Decimal Math Standard:** IEEE 754-2008 Floating-Point Arithmetic rules overridden using `java.math.BigDecimal` with `MathContext.DECIMAL128`.

7.3 Document Revision History & Sign-Off Log

Revision	Release Date	Author / Role	Section(s) Modified	Summary of Key Changes	Approval Status
0.1.0	July 15, 2026	Lead Architect	Sec 1, Sec 2	Initial architecture decision pivoting from microservices to Modular Monolith.	Approved
0.2.0	July 22, 2026	Software Engineer	Sec 2.3, Sec 4	Defined MySQL DDL schema, foreign key cascade constraints, and REST endpoints.	Approved
0.8.0	July 28, 2026	Core Developer	Sec 3, Sec 5	Added mathematical specification for SIP yield formulas and leap-year date clamps.	Approved
1.0.0	July 30, 2026	Lead Architect	Sec 1 – Sec 7	Finalized full SRS baseline including NFR performance targets and error contracts.	Baseline Approved